Progress vivia

Precision

- **Meaning:** Out of all areas the model predicted as *landslide*, how many were actually landslides?
- **Formula:**
  Precision = True Positives / (True Positives + False Positives)
- **Example:** If the model marks 100 regions as landslides but only 80 are correct, precision = 0.8 (or 80%).

Recall

- **Meaning:** Out of all the *actual landslide areas* in the image, how many did the model correctly detect?
- **Formula:**
  Recall = True Positives / (True Positives + False Negatives)
- **Example:** If there are 100 landslide regions and the model detects 90, recall = 0.9 (or 90%).

F1-Score

- **Meaning:** A balance between Precision and Recall — tells how well the model identifies landslides overall.
- **Formula:**
  F1 = 2 × (Precision × Recall) / (Precision + Recall)
- **Example:** If precision = 0.8 and recall = 0.9, F1-score = 0.85 — showing good balanced performance.

👉 In short:

- **Precision** → "How accurate are my landslide predictions?"
- **Recall** → "How many real landslides did I find?"
- **F1-Score** → "Overall balance between the two."

Conv (Convolution Layer)

Full form: Convolution
Think of it as: A feature extractor.
What it does:

- It slides a small window (called a **filter** or **kernel**) over the image.
- Each filter looks for a specific pattern — like edges, color changes, textures, or shapes.
- The output is a **feature map** that shows *where* that pattern appears in the image.

📸 **Example:**
If a filter is trained to detect edges, the convolution output will highlight all edges in the image.

In short:
Conv layer = Detects features like edges, corners, textures.

⚡ 2. ReLU (Rectified Linear Unit)

Full form: Rectified Linear Unit

Think of it as: A decision maker that keeps useful signals.

What it does:

- It changes all negative values to **0** and keeps positive values the same.
  $f(x) = max(0, x)$
- This helps the network learn *non-linear* patterns (complex relationships).

📉 **Example:**
If convolution gives [-5, 2, -3, 6],
ReLU turns it into [0, 2, 0, 6].

In short:

> ReLU = Removes useless negative signals → ma training faster & more stable.

3. Max Pool (MaxPooling)

Full form: Max Pooling Layer
Think of it as: Zooming out and keeping the most important detail.
What it does:

- It looks at small blocks (like 2×2) in the feature map.
- From each block, it keeps only the **maximum (strongest)** value.
- This reduces the image size by half but keeps key information.

📏 **Example:**
Input block:

```csharp
[1, 3]
[2, 4]
```

MaxPooling (2×2) → keeps only the **4**.
Benefits:

- Reduces computation
- Focuses on strong features

- Helps make model more general (less overfitting)

In short:
MaxPool = Shrinks the image while keeping strongest features.

4. Conc (Concatenate / Skip Connection)

Full form: Concatenate
Think of it as: Merging two information streams.
What it does:

- In U-Net, we take a feature map from the **encoder** (left side) and **combine** it with the upsampled feature from the **decoder** (right side).
- This helps recover the original image details lost during downsampling.

📚 **Example:**
Encoder output → [Fine details]
Decoder output → [Coarse structure]
Concatenate → [Fine + Coarse = precise segmentation]

In short:
Concatenate = Combine encoder and decoder info to get sharp, accurate results.

Step	Layer	Purpose
1️⃣	**Conv**	Extract features (edges, patterns)
2️⃣	**ReLU**	Keep only positive activations
3️⃣	**MaxPool**	Reduce image size (zoom out)
4️⃣	**Concatenate**	Combine features from encoder and decoder

Simple Analogy:

Think of U-Net as a **painter**:

- 🎨 **Conv** = observes details of the picture.
- 💡 **ReLU** = keeps only the bright/important parts.
- 🔍 **MaxPool** = zooms out to see the full picture.
- 🔗 **Concatenate** = brings back fine details while redrawing.

What is U-Net?

U-Net is a **deep learning model** used for **image segmentation** — it finds *where* something is in an image (like a landslide region).
It has two main parts:
👉 **Encoder** (understands the image)

👉 **Decoder** (rebuilds the image with predictions)

🧩 1️⃣ Encoder – "Understanding the Image"

- The **encoder** compresses the image and learns what's inside it (like shapes, textures, slopes).
- It reduces image size step by step but captures more meaningful patterns.

Each encoder block has:

- **Conv (3×3):** Finds patterns (edges, colors, slopes).
- **ReLU:** Keeps only useful positive signals.
- **Dropout:** Randomly turns off a few neurons to avoid overfitting.
- **MaxPool (2×2):** Shrinks the image to focus on main features.

Filters: 16 → 32 → 64 → 128 → 256

As we go deeper, filters increase — model learns more complex patterns (like "this texture looks like a landslide").

📌 *Example:* Early layers → edges; deeper layers → shape of a landslide.

💎 2️⃣ Bottleneck – "Deep Thinking Zone"

- It's the center part of U-Net — the **most compressed** and **intelligent** layer.
- It mixes all learned features together and understands overall context.

Structure: Conv → ReLU → Conv → ReLU → (Dropout optional)

📌 *Example:* Knows that "this brown sloped area with vegetation loss likely indicates a landslide."

🪄 3️⃣ Decoder – "Rebuilding with Prediction"

- The decoder **upsamples** (enlarges) the image back to its original size.
- It uses **skip connections** to bring back fine details from the encoder.

Each decoder block does:

- **Upsampling:** Makes image size larger again (×2).
- **Concatenate (skip connection):** Combines details from encoder to keep fine boundaries.
- **Conv + ReLU:** Mix and refine the combined features.

📌 *Why skip connections?*

Without them, the image would look blurry — skip connections bring back details like "edges of landslide".

🎯 4️⃣ Output Layer – "Final Prediction Map"

- The output layer gives a **binary mask** — each pixel gets a value:
  - 1 → Landslide
  - 0 → No Landslide

Structure: 1×1 Conv → Sigmoid

- **1×1 Conv:** Converts features into one output channel (binary).

- **Sigmoid:** Gives probability (0 to 1) for each pixel.
- **Loss:** Binary Cross-Entropy or Dice loss to train the model.

🌈 5️⃣ Handling Multispectral Input (RGB + NDVI + Slope)

- Normally U-Net takes **3 channels (RGB)**.
- In your case, you'll give **5 channels (RGB + NDVI + Slope)**.
- Just set input_channels = 5 in your model.
- Normalize each channel (scale values between 0 and 1).

📌 *Why use these?*
They add more information — NDVI shows vegetation, Slope shows terrain steepness — both useful for detecting landslides.

Part	Purpose	Example
Encoder	Learn features & reduce size	"What patterns exist?"
Bottleneck	Combine features	"Is this region a landslide?"
Decoder	Upsample & reconstruct	"Where exactly is the landslide?"
Output	Create mask	"This pixel = landslide or not"

🧩 Step 4: Model Compilation

- **Optimizer:** Adam – adjusts learning rate automatically for faster convergence.
- **Loss Function:** Binary Cross-Entropy – measures error between predicted and actual masks.
- **Metrics:** Accuracy, Precision, Recall, F1-Score – used to monitor model performance.

⚙️ Step 5: Model Training

- Train model for **100 epochs** with **batch size = 16**.
- Use validation data to monitor performance.
- Apply **ModelCheckpoint** to automatically save the **best model** based on highest F1-Score.

📊 Step 6: Evaluation & Visualization

- Evaluate model on validation dataset to get **loss, accuracy, precision, recall, and F1-score**.
- Plot training vs. validation **loss and accuracy curves**.
- Visualize **predicted masks** alongside input images (RGB, NDVI, Slope) to check results.

💾 Step 7: Model Saving

- Save the **best-performing model** as → best_model.h5.
- Save the **final trained model** for deployment as → model_save.h5.

🕐 Epoch

- **Definition:**
  One **epoch** means the model has seen **all training images once**.
- During each epoch, the model goes through your entire dataset and adjusts its weights (learns).
- More epochs = more learning cycles.

📘 **Example:**
If you have 1,000 training images and train for **100 epochs**,
→ The model will see those 1,000 images **100 times** (each time improving a bit).

✅ **In short:**
Epoch = 1 complete round of training on the full dataset.

📦 Batch Size

- **Definition:**
  The number of training images the model processes **at once** before updating its weights.
- Instead of loading all images together (which needs too much memory), we split them into **batches**.

📘 **Example:**
If you have 1,000 images and batch size = 16,
→ Model trains on 16 images at a time, then updates its weights.
→ After 1,000 / 16 = ~63 updates, one epoch is complete.

✅ **In short:**
Batch size = How many images are processed before one update.

Term	Meaning	Example
Epoch	One full pass through dataset	100 epochs = 100 passes
Batch Size	Mini-group of images used at a time	16 images per update
Why important?	Affects training speed, memory use, and model stability	Too large → slow learning; Too small → unstable learning

What is an Activation Function?

- An **activation function** decides **whether a neuron should "fire" or not** — basically, it adds *intelligence* to your neural network.

- It helps the model **learn complex patterns** (not just straight lines).
- Without activation functions, your model would behave like a simple linear calculator — it couldn't learn curves, edges, or abstract shapes.

🧠 **In simple words:**
Activation functions help the model understand **non-linear** patterns like "landslide shape" or "texture" instead of just color intensity.

🧩 Types of Activation Functions (used in your project):

1️⃣ ReLU (Rectified Linear Unit)

- Formula: f(x) = max(0, x)
- Negative values become **0**, positive values stay the same.
- Makes training faster and helps focus on useful signals.

📘 **Example:**
If a neuron outputs [-3, 2, -5, 4] → ReLU gives [0, 2, 0, 4].
✅ **Used in:** Encoder and Decoder layers
✅ **Purpose:** Learn meaningful features (edges, slopes, etc.)

2️⃣ Sigmoid Function

- Formula:

$$f(x) = \frac{1}{1 + e^{-x}}$$

- It **converts output values into probabilities** between **0 and 1**.
- Ideal for **binary classification** (like *Landslide* vs *No Landslide*).

📘 **Example:**
If Sigmoid output =

- 0.95 → Very likely landslide
- 0.10 → Very unlikely landslide

✅ **Used in:** Output layer of U-Net
✅ **Purpose:** Gives pixel-wise probability of landslide presence.

Function	Acts Like	Meaning
ReLU	Filter	Keeps only positive signals (important patterns)
Sigmoid	Probability gate	Converts output to "how likely is this a landslide"

🧠 What is Encoder–Decoder?

Think of it like a **translator** —

The **encoder** understands the image (compresses it),
and the **decoder** rebuilds it (expands it back) to show *where* the landslide is.

🏗️ 1️⃣ Encoder (Contracting Path)

Purpose:
To **analyze and extract important features** from the input image.
What it does:

- Takes the input (RGB + NDVI + Slope).
- Passes it through multiple layers that **detect patterns** like edges, slopes, soil texture, etc.
- **Reduces image size** step by step but **increases feature depth** (more filters).
- Learns **what** is in the image (landslide vs non-landslide).

Common Layers:
👉 Conv → ReLU → Dropout → MaxPooling
Example:
If the original image is 256×256, after a few encoder blocks, it might become 16×16 — smaller but rich in learned features.
In short:
Encoder = "Learns and compresses meaningful information from the image."

🧩 2️⃣ Decoder (Expanding Path)

Purpose:
To **reconstruct the image** and **predict pixel-wise segmentation mask**.
What it does:

- Takes the compressed info from the encoder (bottleneck).
- **Upsamples** (increases image size) to get back to original resolution.
- **Concatenates (skip connections)** with encoder outputs to bring back lost details.
- Generates the **final segmentation mask** showing landslide regions.

Common Layers:
👉 Upsampling (or ConvTranspose) → Concatenate → Conv → ReLU
In short:
Decoder = "Rebuilds the image to show *where* the landslide is."

🔗 Skip Connections (Bridge Between Encoder & Decoder)

- They **connect matching layers** from encoder to decoder.
- Help restore fine details (boundaries, edges) lost during compression.

📘 Example:
Without skip connections → blurred boundaries.
With skip connections → crisp and accurate landslide borders.

🧩 Simple Analogy:

Part	Think of as	Function

Encoder	**Student learning**	Compresses image to learn patterns
Decoder	**Student explaining**	Expands image to show what was learned
Skip Connection	**Notes linking both**	Keeps details for better explanation